



CAP6412

Advanced Computer Vision

Spring 2026

Yogesh S Rawat

yogesh@ucf.edu

UCF Global 223

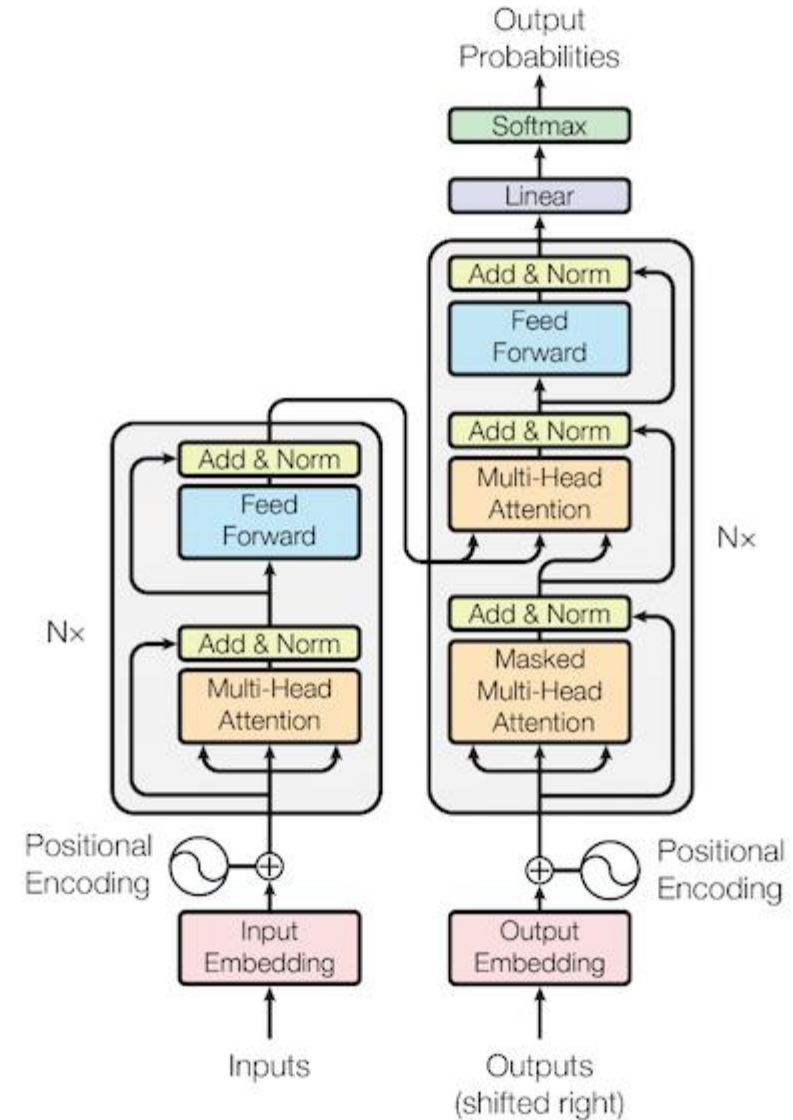


Transformer

Lecture 2

Content

- Basics
 - What is Transformer?
 - Self-Attention
 - Query, Key, Value
 - Position encoding
 - Encoder-Decoder
- BERT
- Vision Transformers
 - ViT
 - SWIN



Transformer

- Used for modeling long dependencies between input sequence elements
- Supports parallel processing of sequence
 - As compared to RNN (e.g. LSTM)
- Allows processing multiple modalities
 - (e.g., images, videos, text and speech) using similar processing blocks
- Typically, pre-trained using pretext tasks on largescale (unlabeled) datasets
- Demonstrates excellent scalability to very large networks and huge datasets.
- **GPT-4**

Natural Language Processing

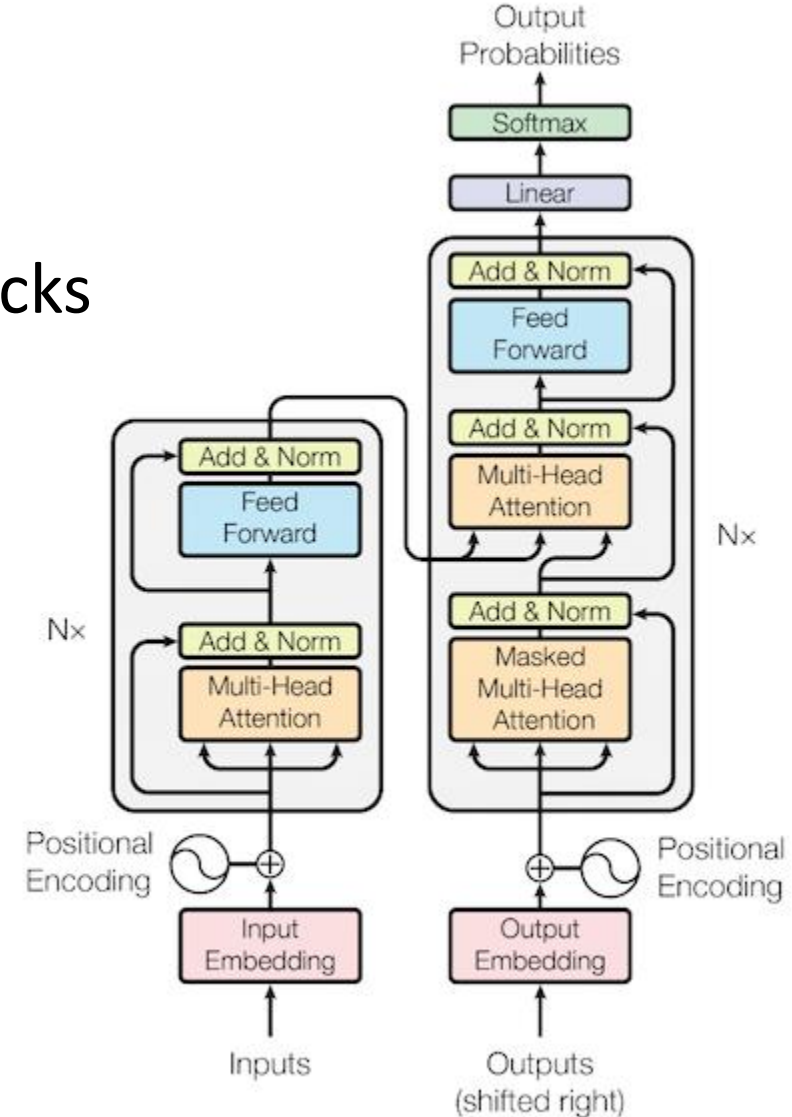
- BERT (Bidirectional Encoder Representations from Transformers),
- GPT (Generative Pre-trained **Transformer**) v1-4,
- RoBERTa (Robustly Optimized BERT Pre-training)
- T5 (Text-to-Text Transfer Transformer)
- Gemini
- LLaMA
- Most recent LLMs and VLMs

Vision Applications

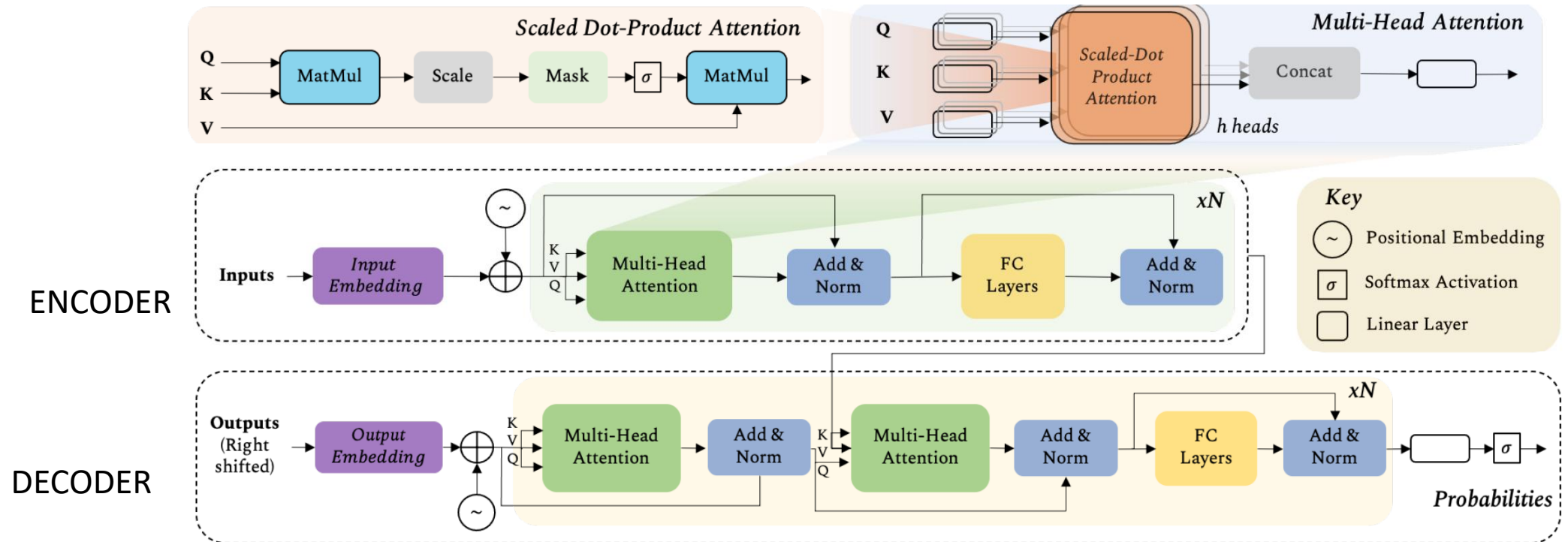
- Recognition tasks
 - e.g., image classification, object detection, and segmentation
- Generative modeling, multi-modal tasks
 - e.g., visual-question answering, visual reasoning, and visual grounding
- Video processing
 - e.g., activity recognition, frame prediction
- Low-level vision
 - e.g., image super-resolution, image enhancement, and colorization)
- 3D analysis
 - e.g., point cloud classification and segmentation)

Transformer Basics

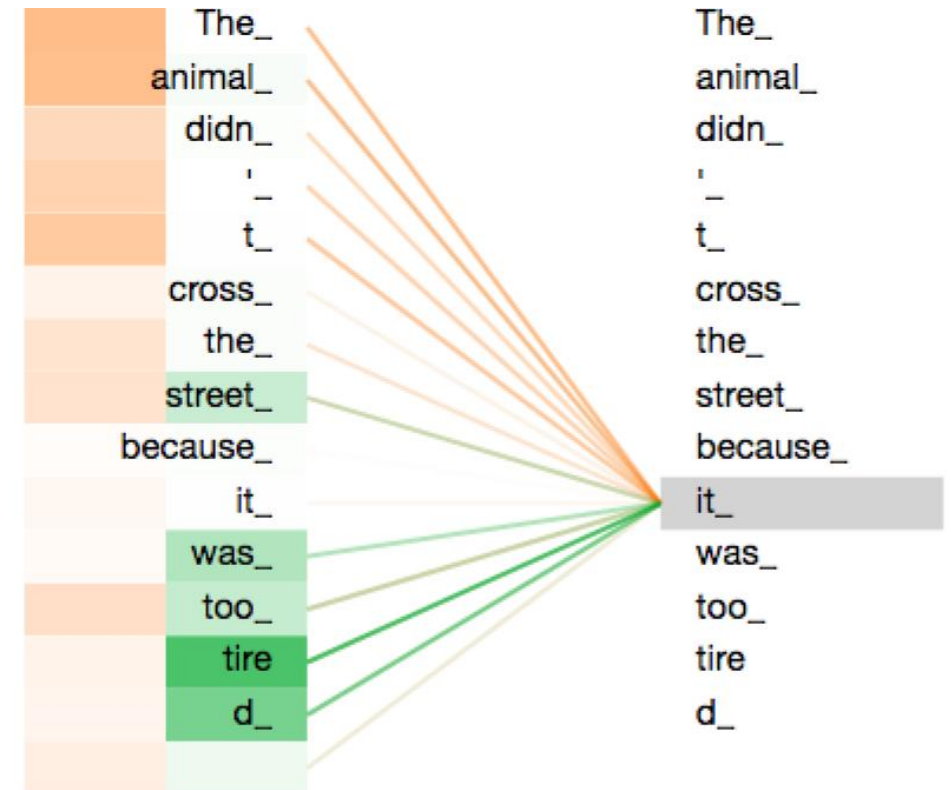
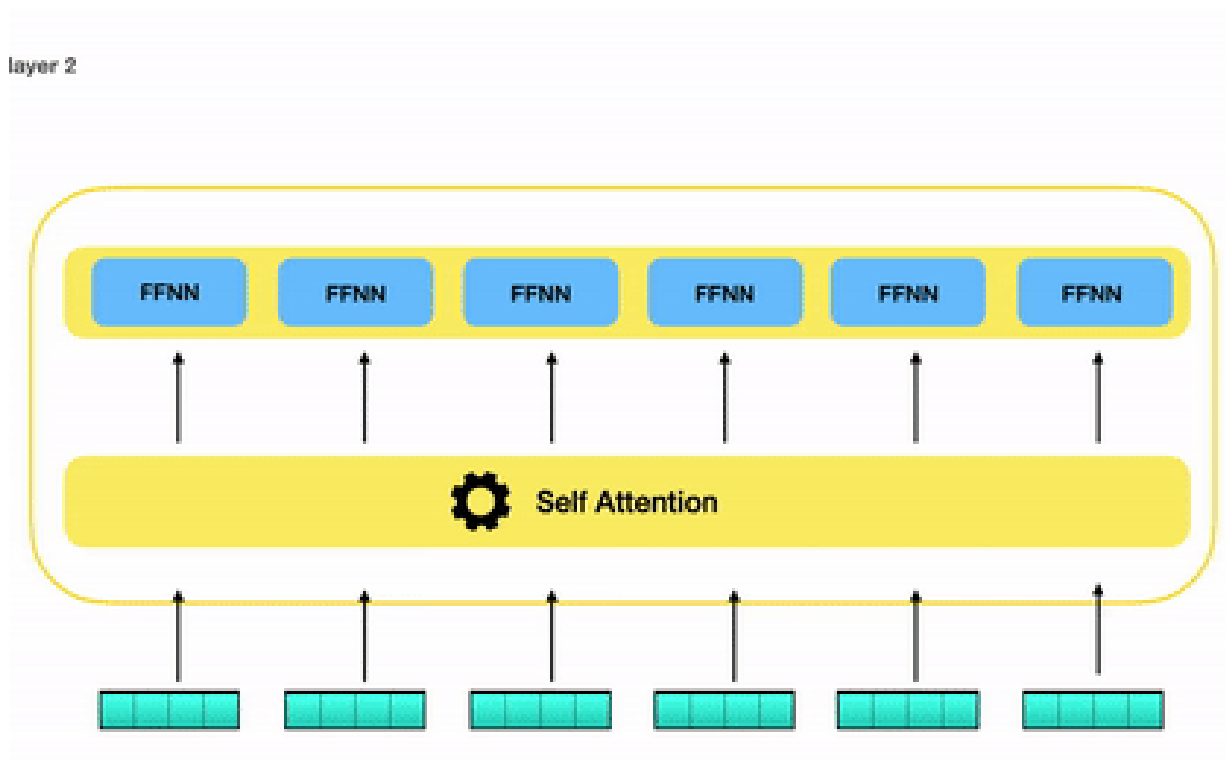
- It consists of Encoder and Decoder Blocks
- Main components of each block:
 - Self-Attention
 - Layer Normalization
 - Feed Forward Network



Transformer

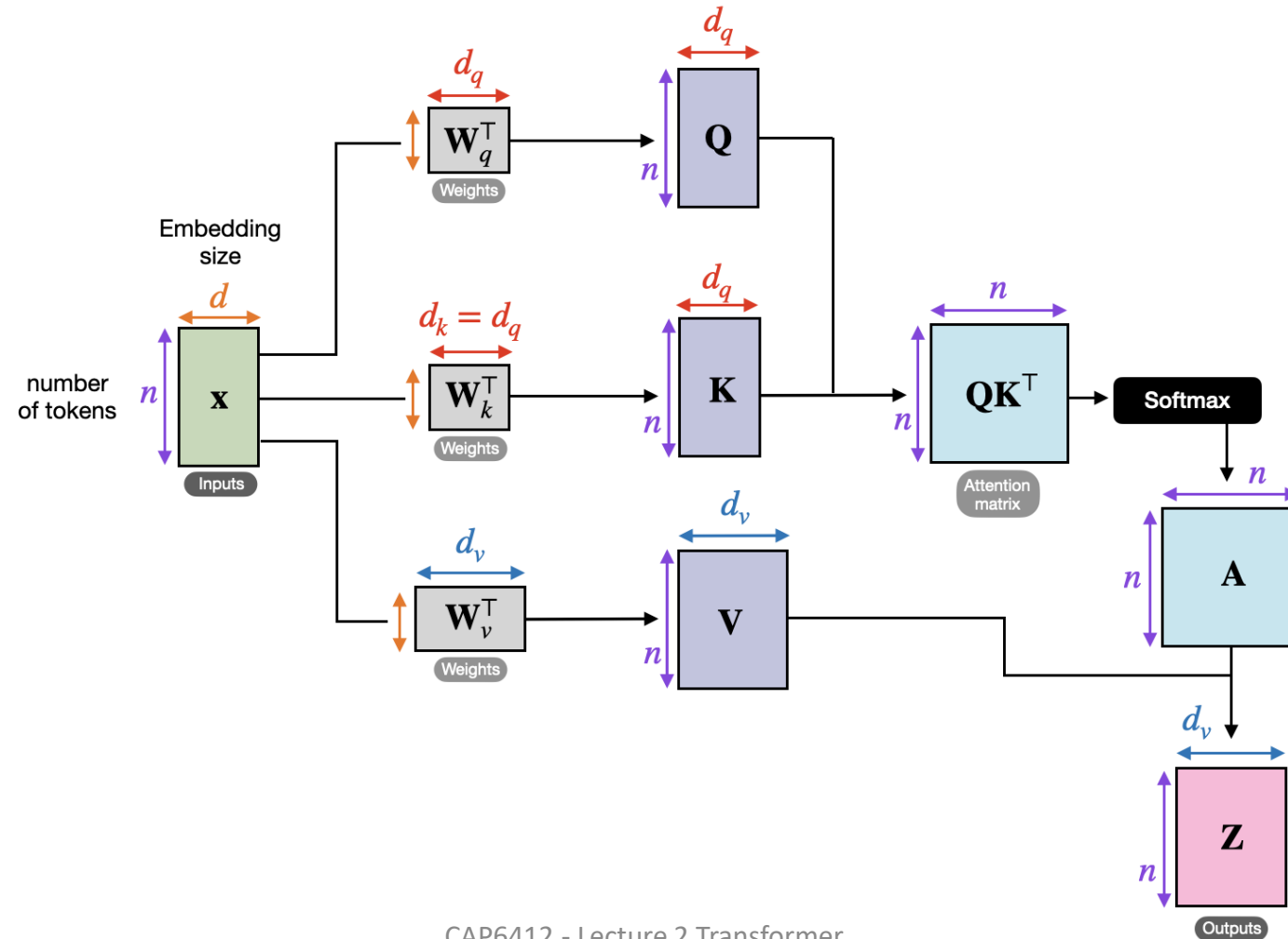


Self-attention



<https://raviteja-ganta.github.io/attention-is-all-you-need-transformers>

Operation in self-attention





Recap

CAP5415

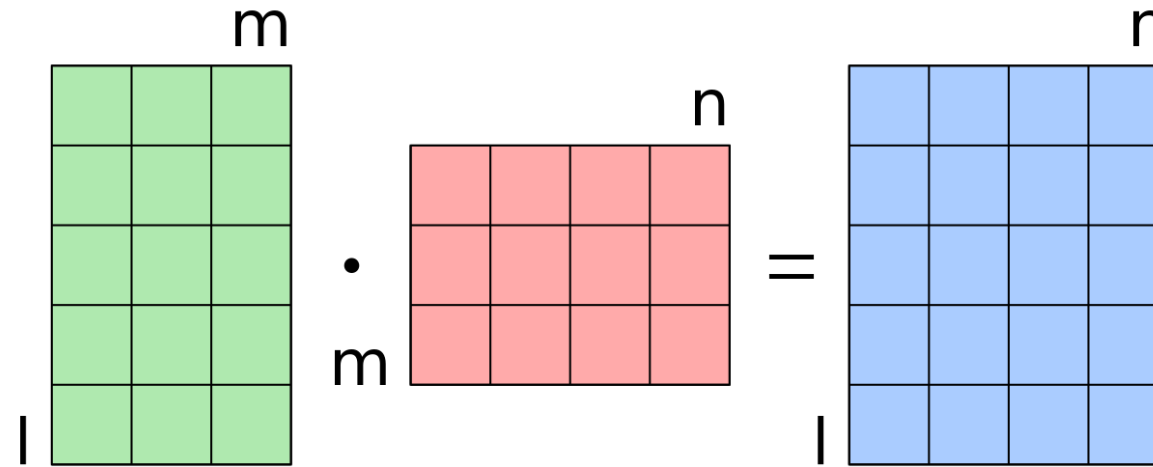
Remember matrix multiplication?

Matrix operation

- Matrix Multiplication – another interpretation (my favorite)
 - Let $\mathbf{a}_i$ denote the i -th column of the matrix $\mathbf{A}$, and
 - $\mathbf{b}_j$ denote the j -th column of the matrix $\mathbf{B}$.
 $\mathbf{A} = [\mathbf{a}_1 \quad \mathbf{a}_2 \quad \cdots \quad \mathbf{a}_n]$, and $\mathbf{B} = [\mathbf{b}_1 \quad \mathbf{b}_2 \quad \cdots \quad \mathbf{b}_m]$
 - The first column of $\mathbf{AB}$
 - Linear combination of all the columns in $\mathbf{A}$

$$[\mathbf{a}_1 b_{11} + \mathbf{a}_2 b_{12} + \cdots + \mathbf{a}_n b_{1n}]$$

Matrix operation



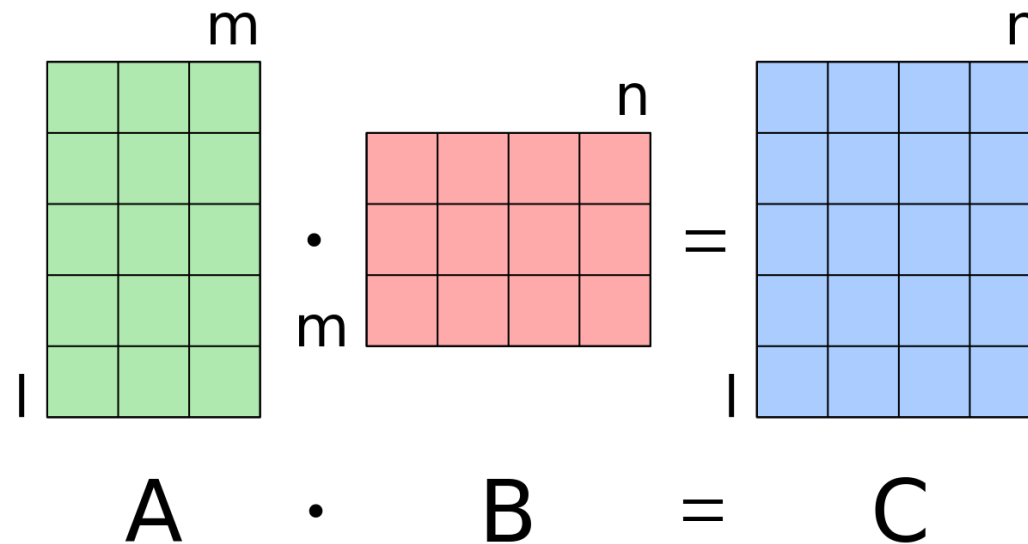
$$A \cdot B = C$$

$$[a_1b_{11} + a_2b_{21} + \dots + a_mb_{m1}]$$

- Similarly, we can get other columns...

Matrix operation

- How about linear combination of all rows of **B**?
 - Each row of **C = AB** is a linear combination of rows of **B**



$$[b_1 a_{11} + b_2 a_{12} + \dots + b_m a_{1m}]$$

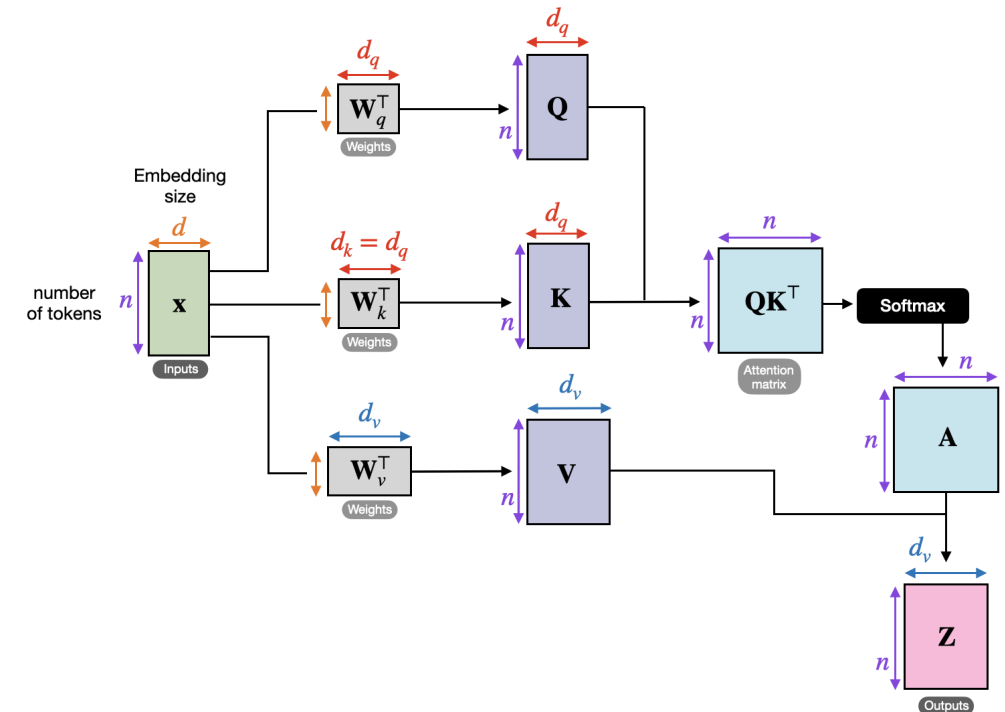
Self-Attention (Matrix Explanation)

Lets denote a sequence of n entities $(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n)$

by $\mathbf{X} \in \mathbb{R}^{n \times d}$,

Queries ($\mathbf{W}^Q \in \mathbb{R}^{d \times d_q}$), Keys ($\mathbf{W}^K \in \mathbb{R}^{d \times d_k}$) and Values ($\mathbf{W}^V \in \mathbb{R}^{d \times d_v}$), where $d_q = d_k$. The input sequence $\mathbf{X}$ is first projected onto these weight matrices to get $\hat{\mathbf{Q}} = \mathbf{X}\mathbf{W}^Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K$ and $\mathbf{V} = \mathbf{X}\mathbf{W}^V$. The output $\mathbf{Z} \in \mathbb{R}^{n \times d_v}$ of the

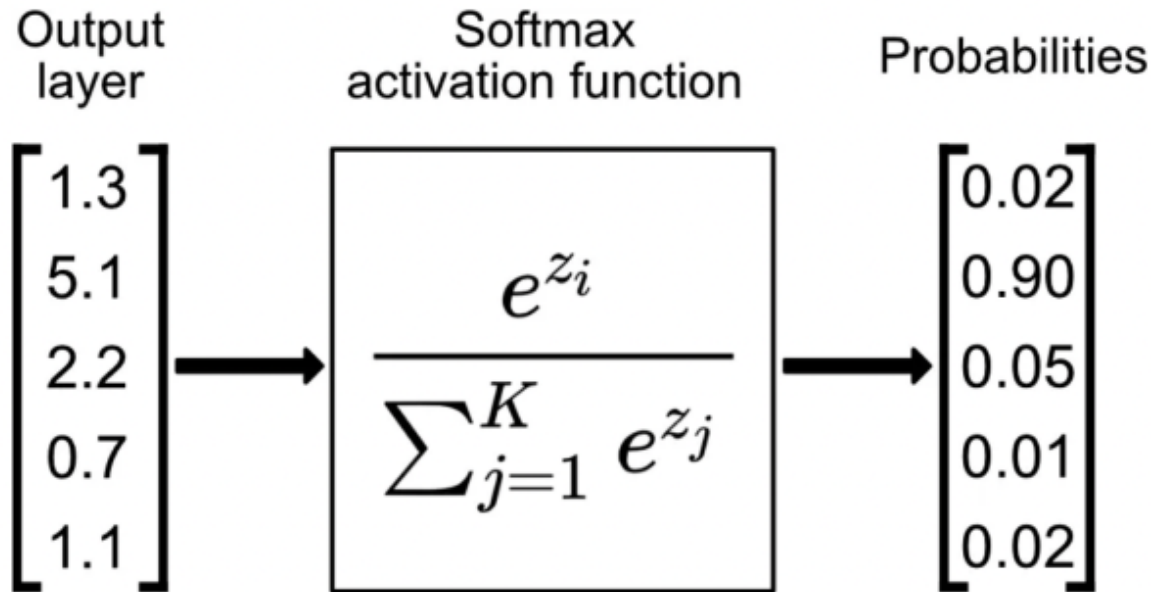
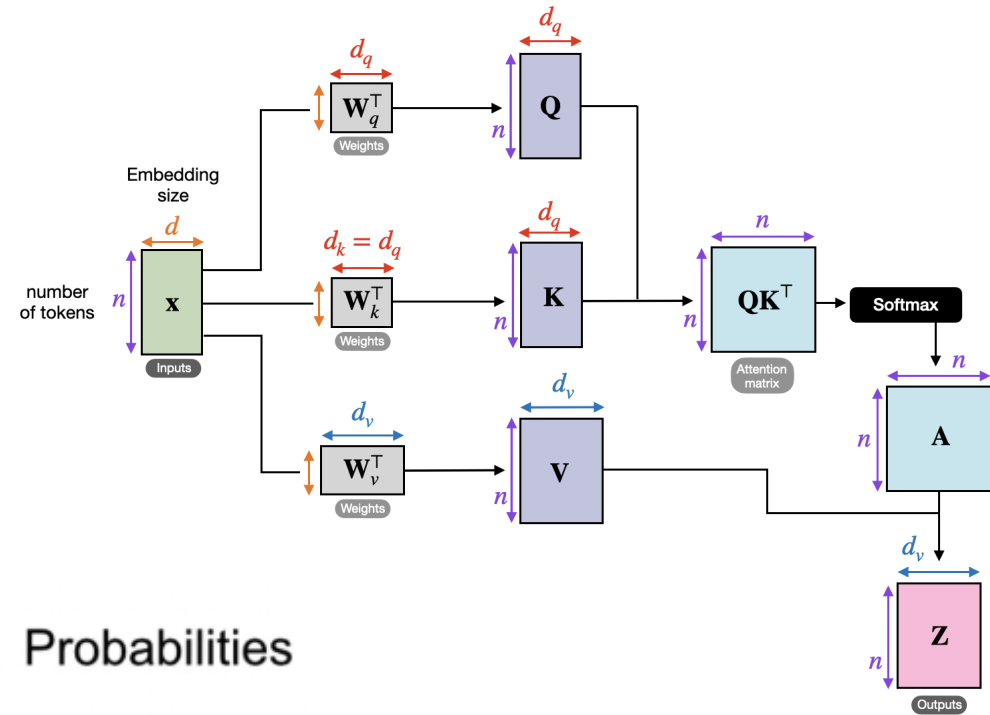
$$\mathbf{Z} = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_q}} \right) \mathbf{V}.$$





Softmax

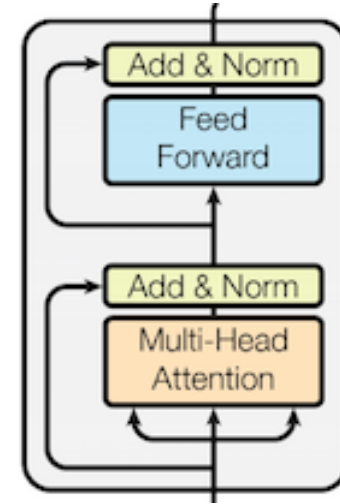
- Done along rows
 - Why?





Multiple heads

- You break each token (K,V,Q) into multiple tokens
 - Along the feature dimension
 - For example, 12 in GPT-2
- Each head processes a segment of the embeddings independently
 - Capturing different syntactic and semantic relationships.
- This design facilitates parallel learning of diverse linguistic features
 - Enhancing the model's representational power.

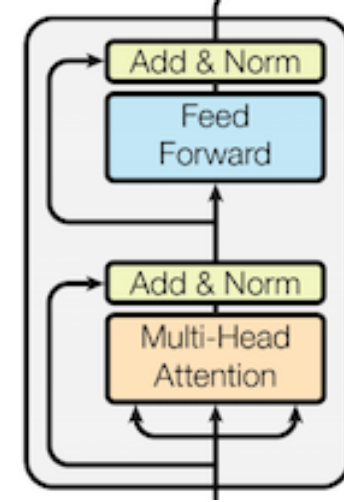


<https://poloclub.github.io/transformer-explainer/>

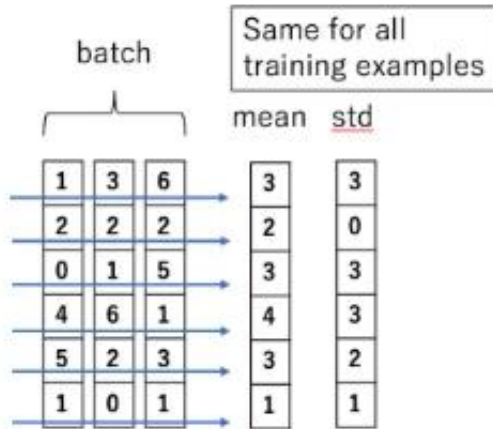


Layer normalization

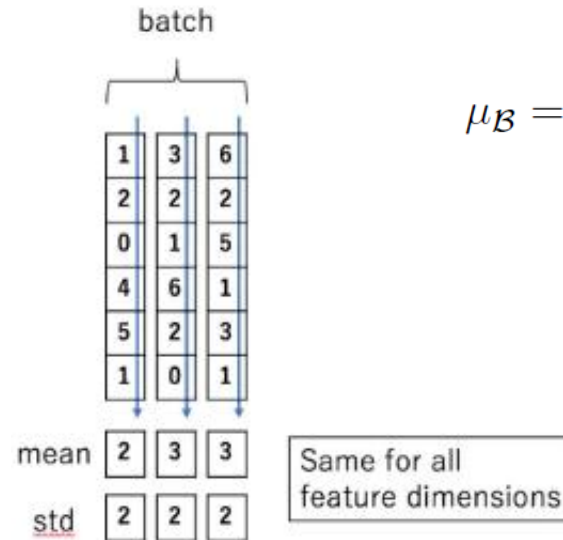
- Layer normalization normalizes the inputs across the features
 - Why not BatchNorm?



Batch Normalization



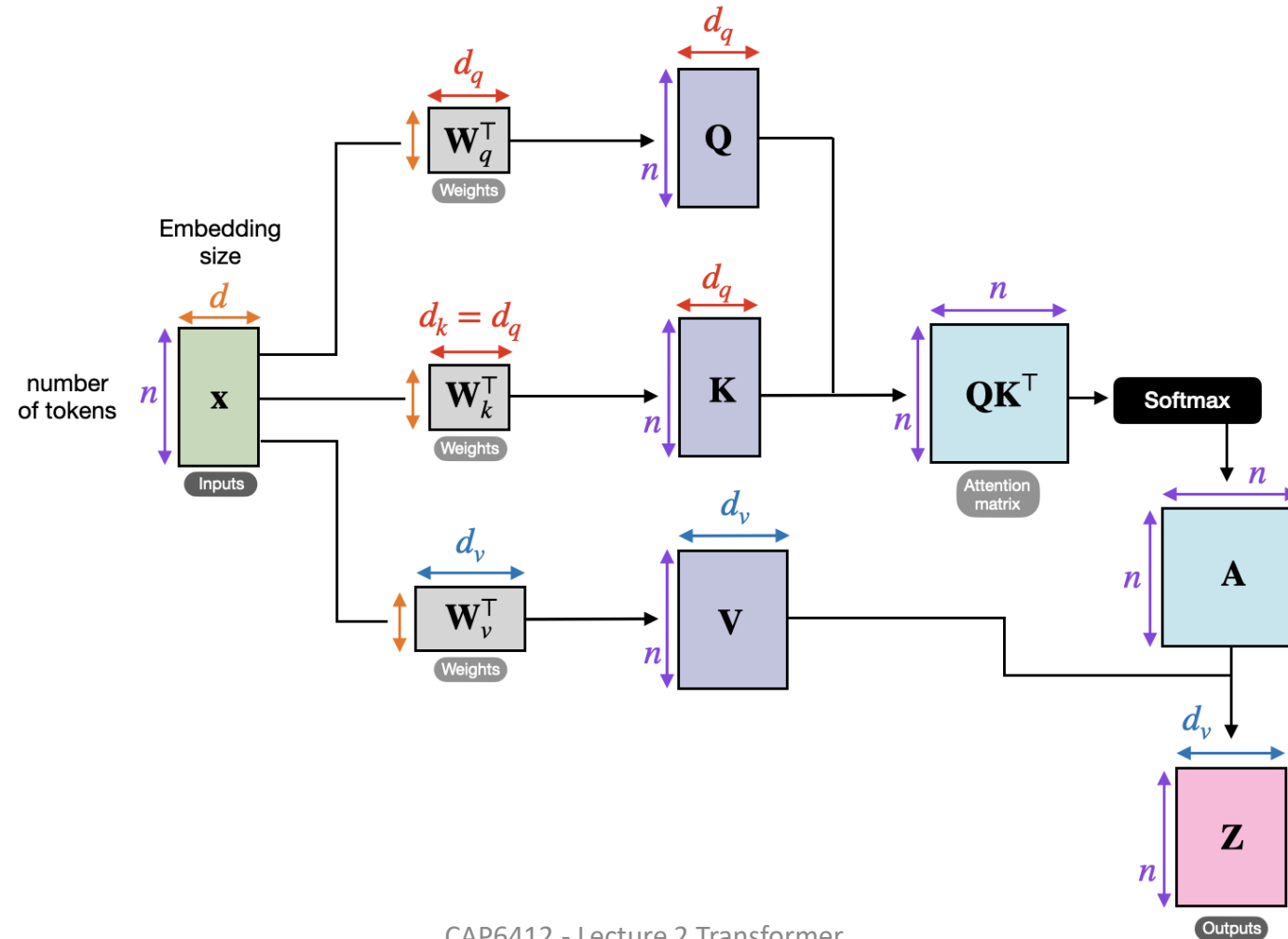
Layer Normalization



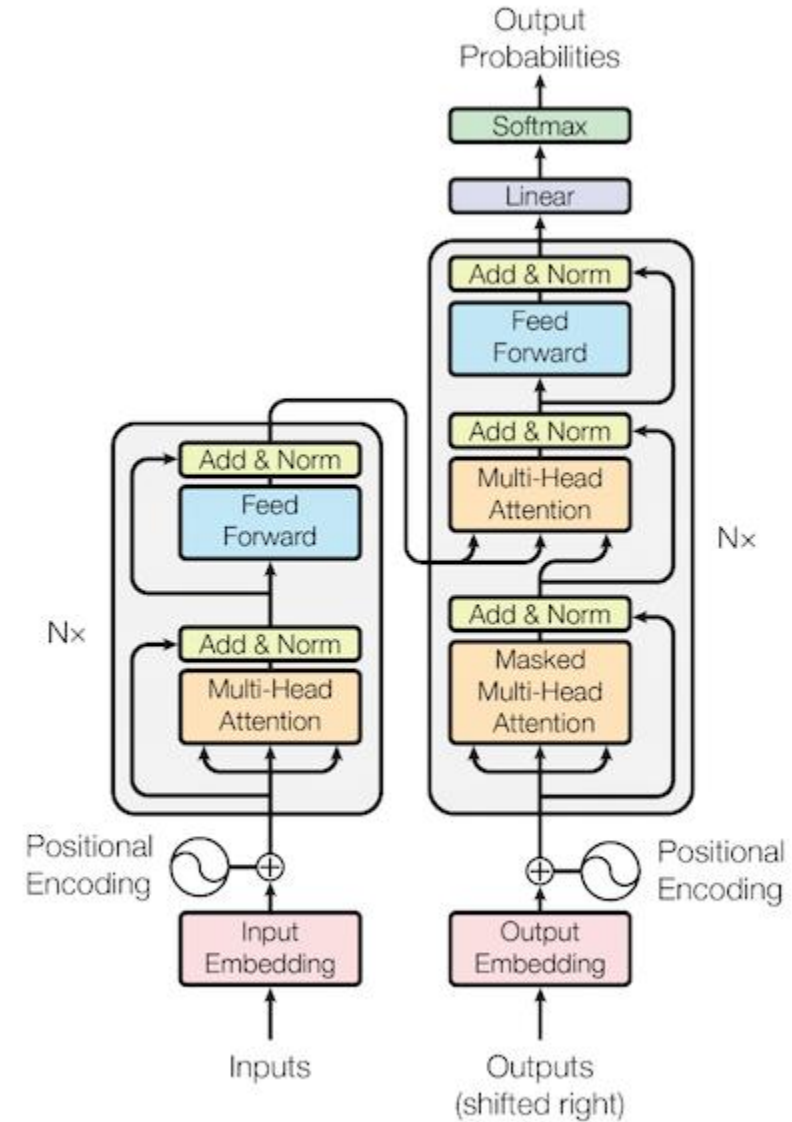
$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i \quad \sigma_{\mathcal{B}} = \sqrt{\frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 + \epsilon}$$

$$\hat{x}_i = \frac{1}{\sigma_{\mathcal{B}}} (x_i - \mu_{\mathcal{B}}) \quad y_i = \gamma \hat{x}_i + \beta$$

Where is learning?

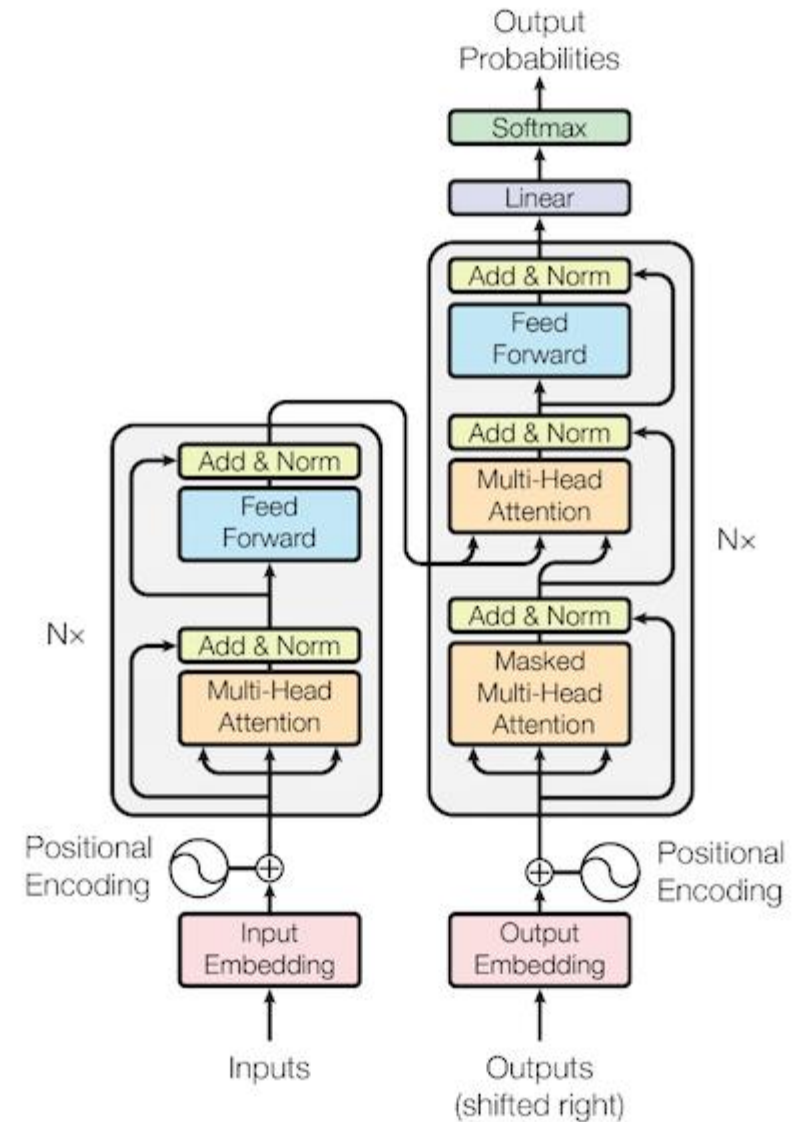


Questions?



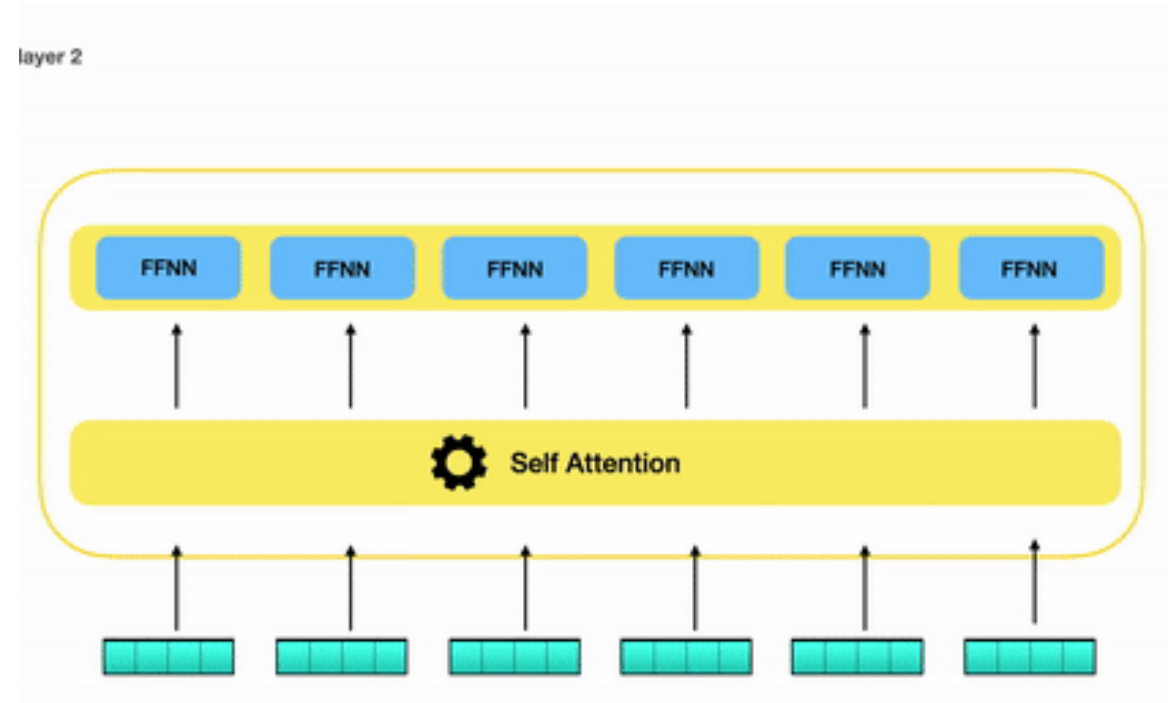
Content

- **Basics**
 - **What is Transformer?**
 - **Self-Attention**
 - **Query, Key, Value**
 - Position encoding
 - Encoder-Decoder
- BERT
- Vision Transformers
 - VIT
 - SWIN



Positional encodings

- Transformer is permutation invariant
- Transformer process tokens in parallel
- Attention mechanism does not take into account
 - The position of a word in the sequence



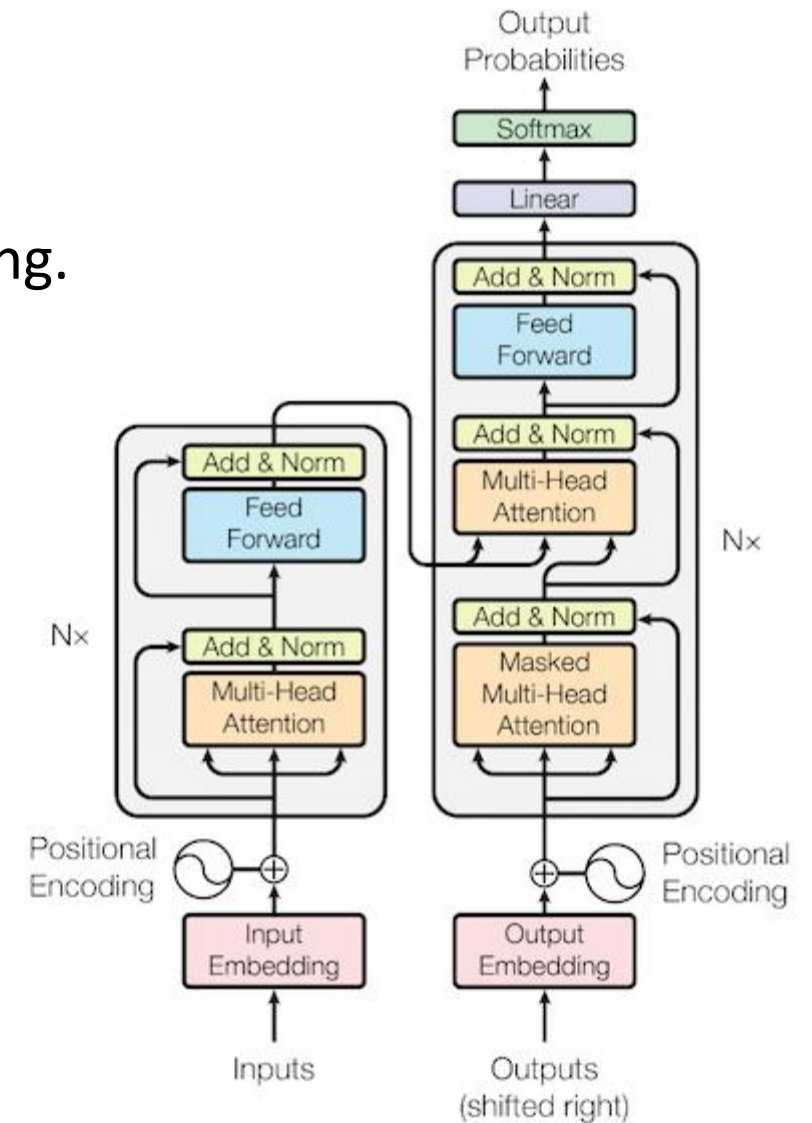
<https://raviteja-ganta.github.io/attention-is-all-you-need-transformers>

Why positional encodings

- “The teacher said the student is smart.”
- “The student said the teacher is smart.”
- “Only she said he was wrong.”
- “She said only he was wrong.”

Positional encodings

- The transformer adds a vector to each input embedding.
- These vectors follow a specific pattern that the model learns
- Learned pattern helps model
 - to determine the position of each word, or
 - the distance between different words.





Position Encoding

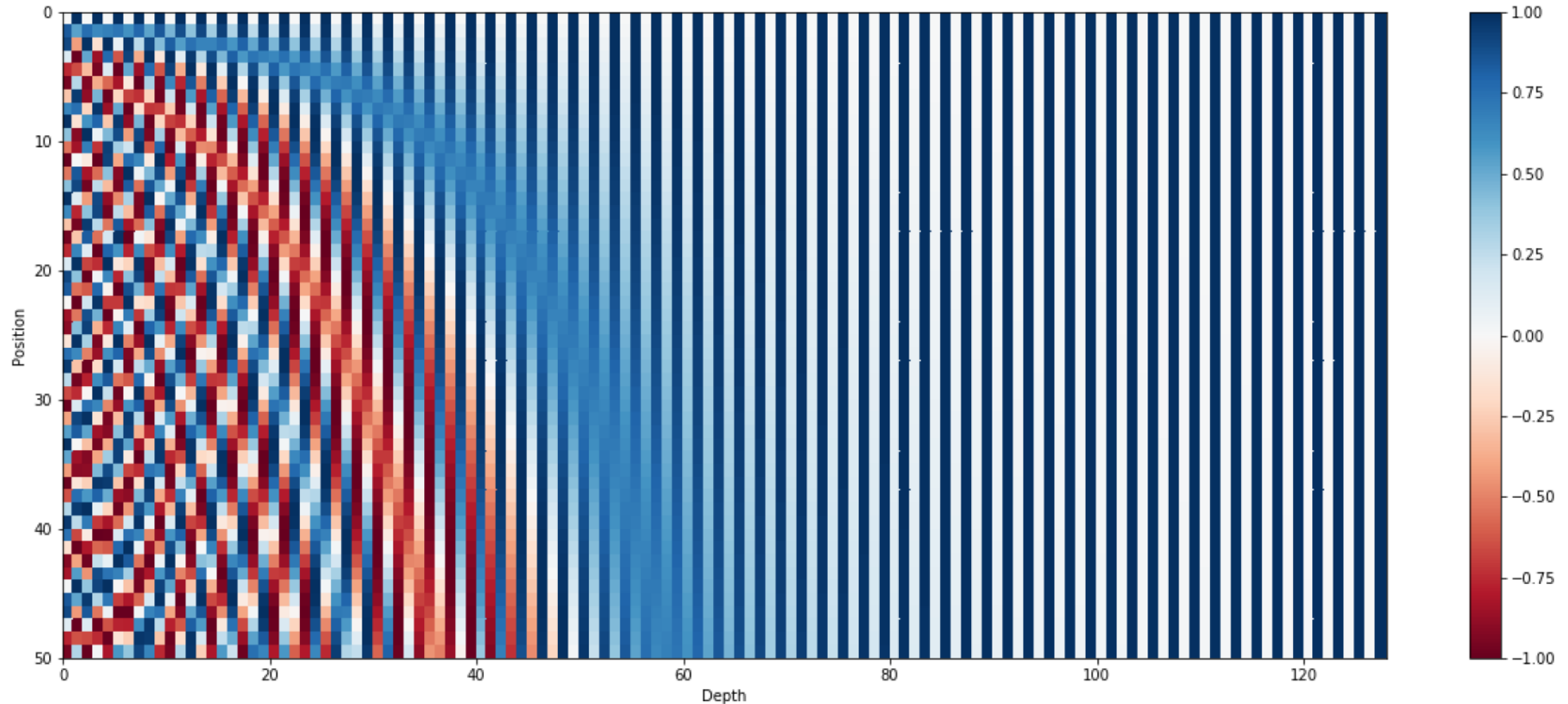
$$\vec{p}_t^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k \cdot t), & \text{if } i = 2k \\ \cos(\omega_k \cdot t), & \text{if } i = 2k + 1 \end{cases}$$

where

$$\omega_k = \frac{1}{10000^{2k/d}}$$

$$\vec{p}_t = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \\ \sin(\omega_2 \cdot t) \\ \cos(\omega_2 \cdot t) \\ \\ \vdots \\ \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}_{d \times 1}$$

Position Encoding





Position Encoding

$$P(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right)$$

$$P(k, 2i + 1) = \cos\left(\frac{k}{n^{2i/d}}\right)$$

Positional Encoding
Matrix with $d=4$, $n=100$

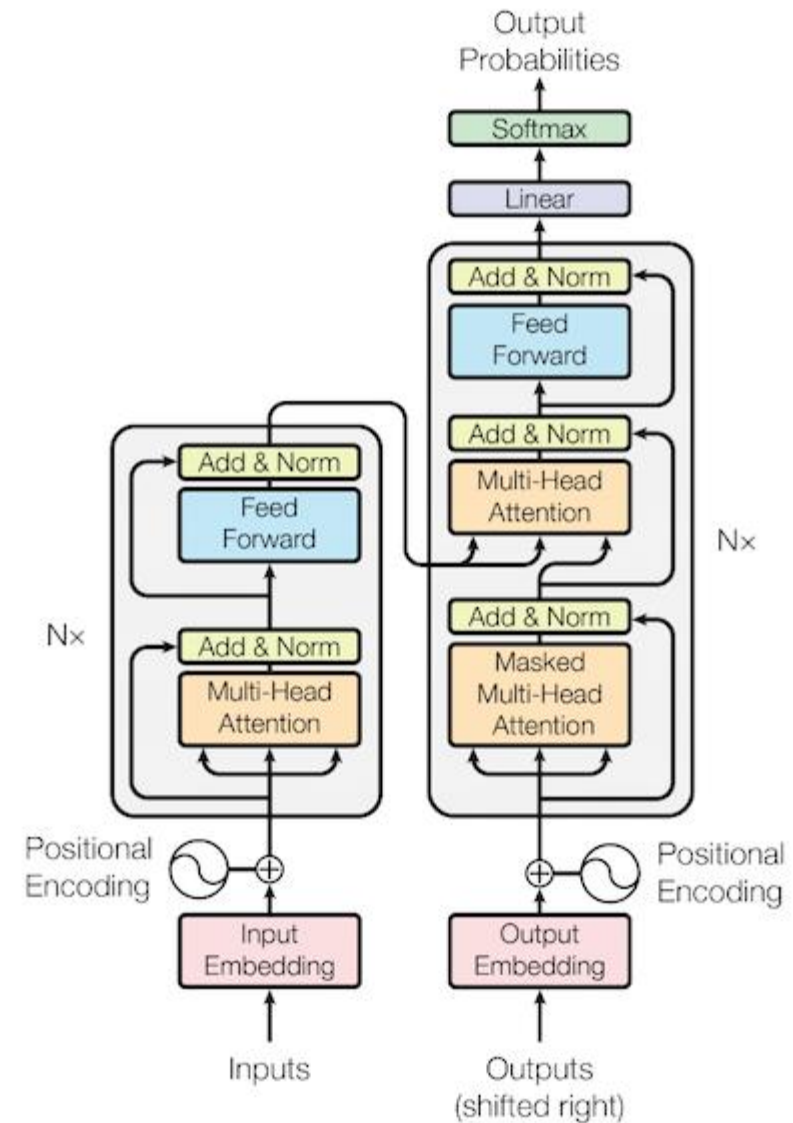
Sequence	Index of token, k	$i=0$	$i=0$	$i=1$	$i=1$
I	0	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	1	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	2	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	3	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

Position Encoding

- Can also be learned
 - Learn like other parameters
- **CoPE** (Complex Positional Encoding)
 - A lightweight method designed to be more flexible
 - Enabling positional encoding based on token context
 - Rather than just absolute distance.
- **RoPE** (Rotary Position Embedding)
 - To better handle long-context sequences
 - By multiplying the attention matrix with a rotation matrix

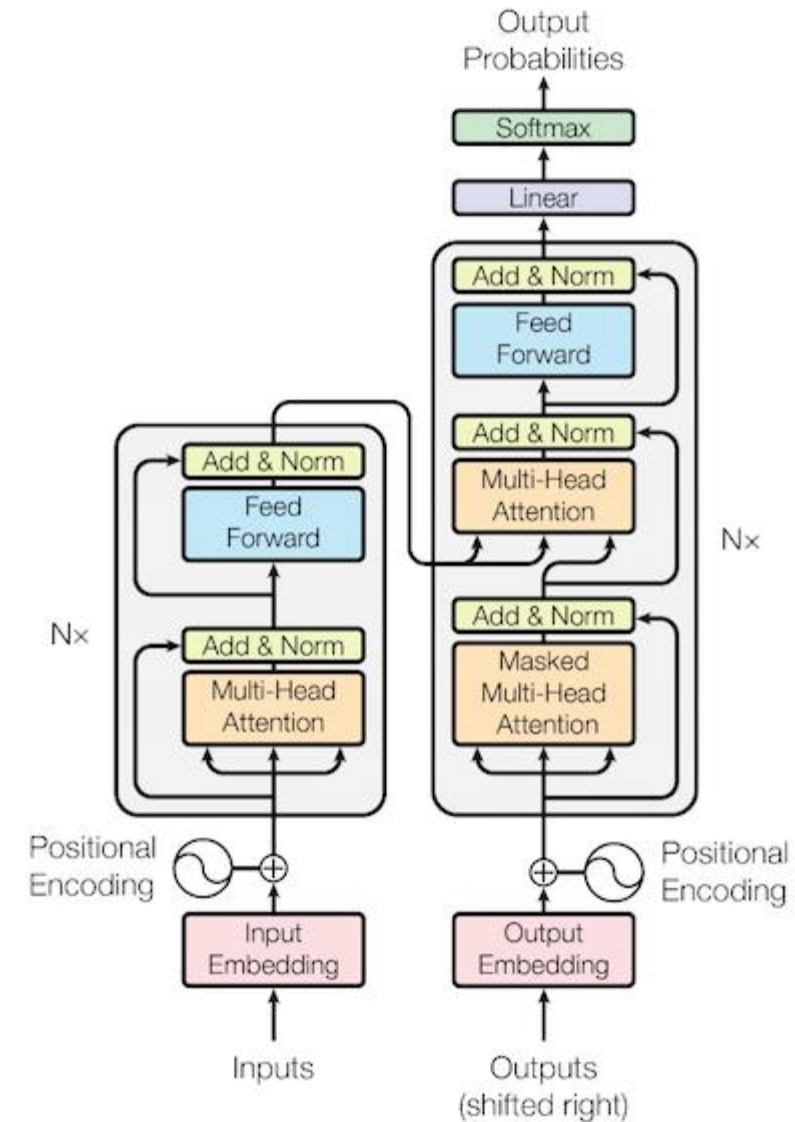
Content

- **Basics**
 - **What is Transformer?**
 - **Self-Attention**
 - Query, Key, Value
 - **Position encoding**
 - Encoder-Decoder
- BERT
- Vision Transformers
 - VIT
 - SWIN



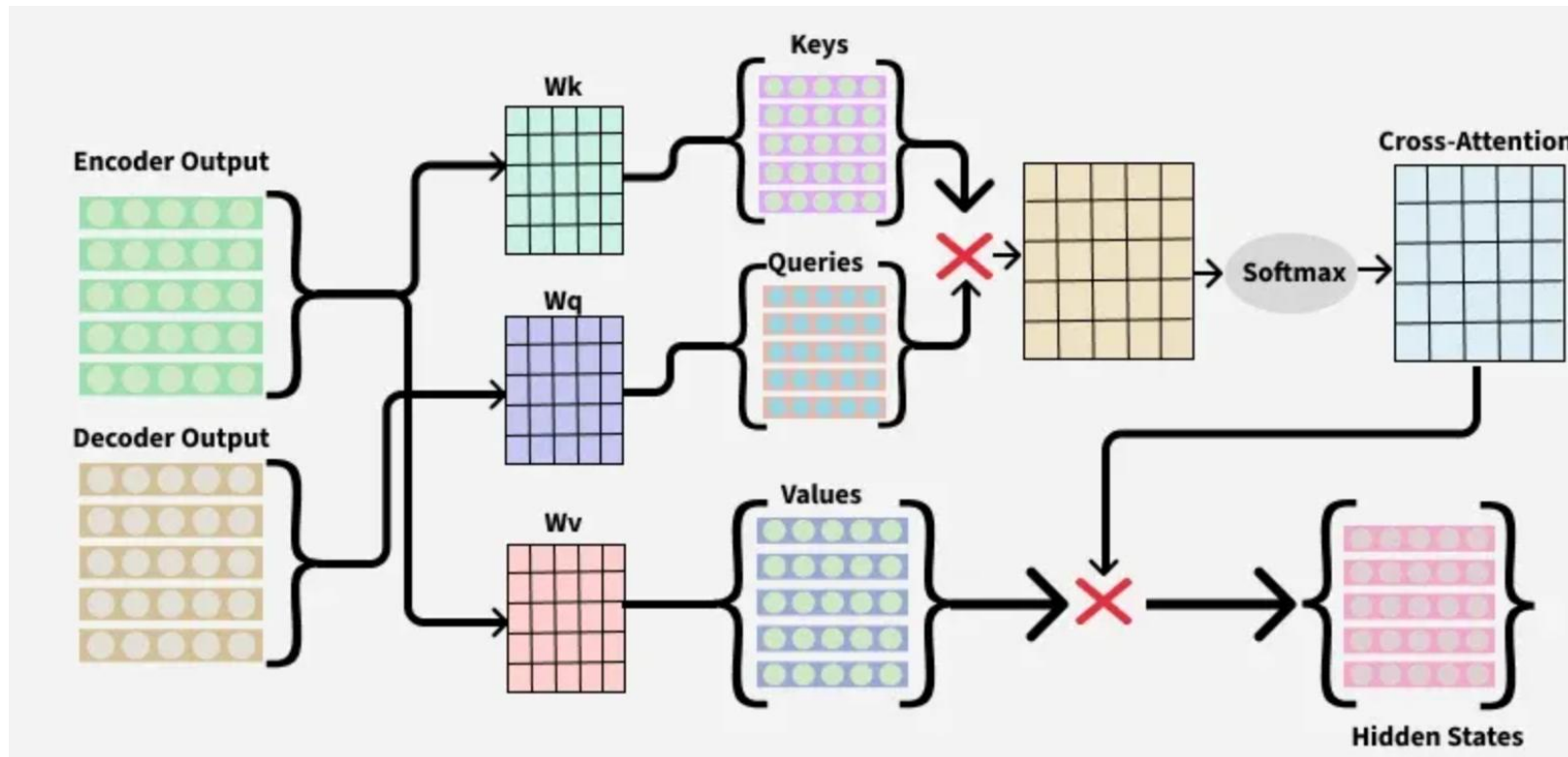
Encoder-decoder

- Output of encoder is used in attention
- The same output is used in all decoder blocks



Encoder-decoder

- Cross-attention





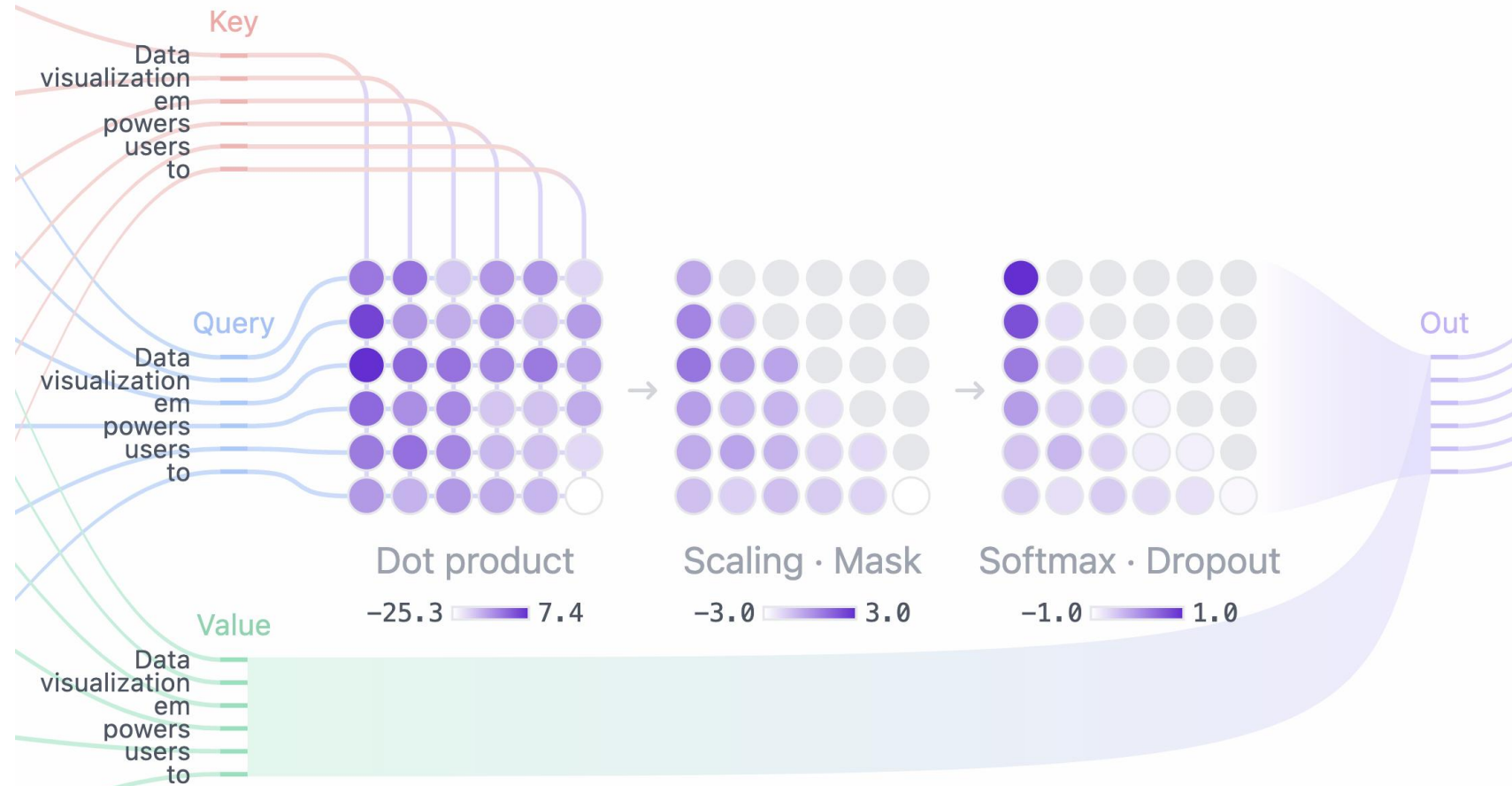
Attention masking

- Decoder is autoregressive
 - Avoid seeing the future

the	cake	was	sour
the	cake	was	sour
the	cake	was	sour
the	cake	was	sour



Attention masking



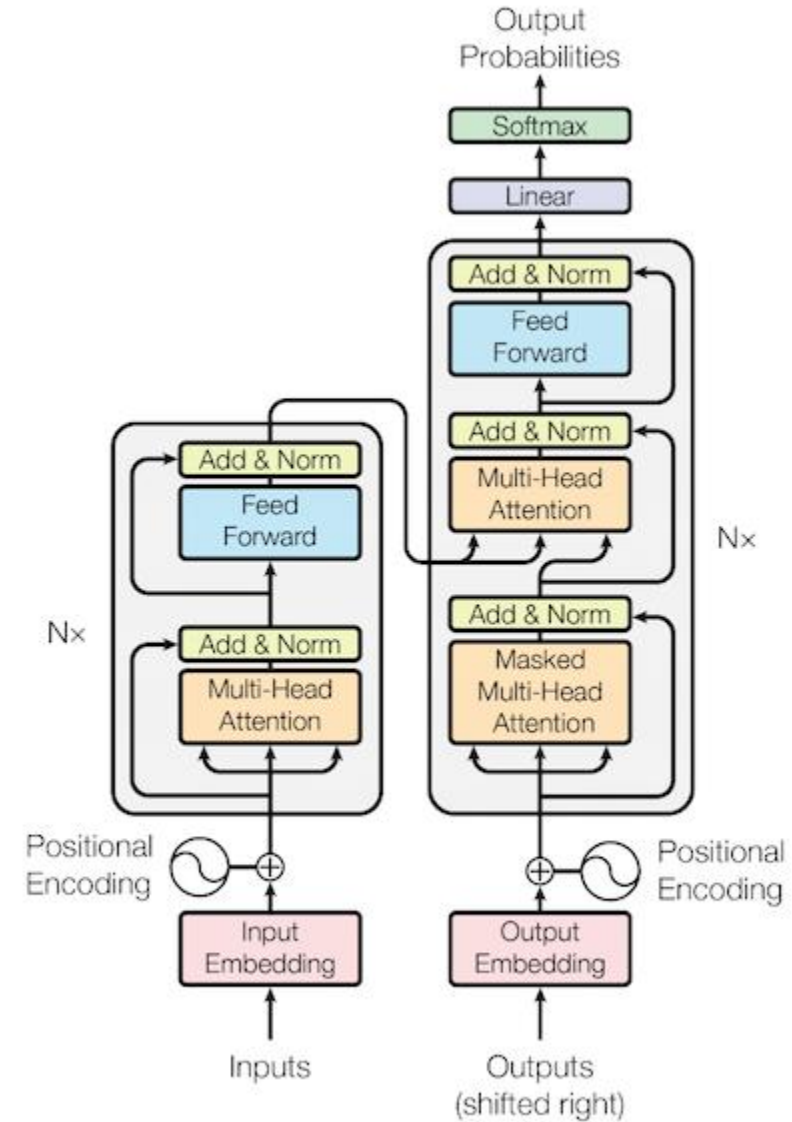
<https://poloclub.github.io/transformer-explainer/>

1/27/2026

CAP6412 - Lecture 2 Transformer

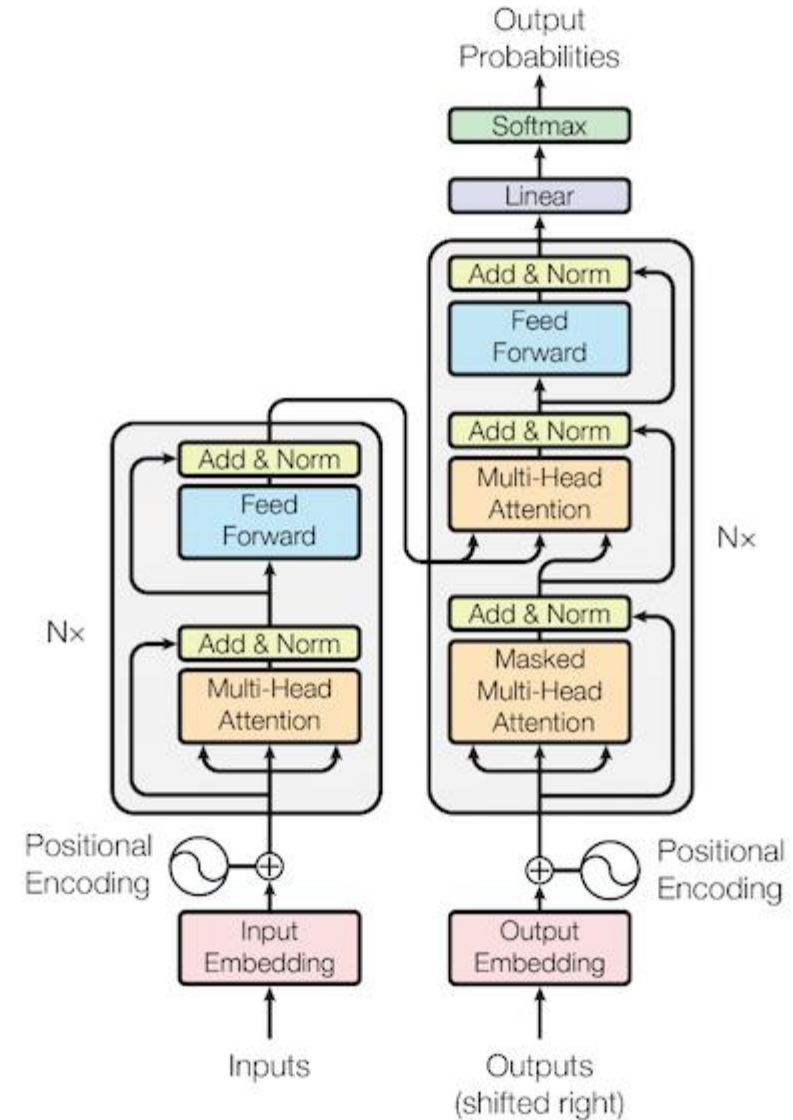
33

Questions?



Content

- **Basics**
 - What is Transformer?
 - Self-Attention
 - Query, Key, Value
 - Position encoding
 - Encoder-Decoder
- BERT
- Vision Transformers
 - ViT
 - SWIN



Demo

- <https://poloclub.github.io/transformer-explainer/>

Questions?

Thank you!

Slides credit: Mubarak Shah